

started at system boot up and exits when there are no more jobs to run. Upon start-up, Anacron reads the list of configured jobs from the Anacron table. For each job, it checks whether the current job has been executed in the last 'n' days. If not, Anacron waits for the number of minutes specified as the delay parameter in the Anacron table, and starts executing the job. After this is done, it records the job execution date in a special timestamps file. At the next system boot up cycle, this timestamps file is checked to decide whether there is a need for a particular job to be executed or not. Anacron makes sure that only one instance of a job runs at a time by using a locking mechanism. After executing all jobs, Anacron exits.

Understanding the Anacron table

Now that we have understood the need for Anacron and its lifecycle, let us explore the format of the Anacron table. In the Anacron table, each field is separated by a space or a tab character. Shown below is the format of the Anacron table:

```
{period} {delay} {job-identifier} {command(s)}
```

OR

```
{@period_name} {delay} {job-identifier} {command(s)}
```

Table 1 describes each field of the Anacron table.

Table 1

Field	Description
period	This is a numeric value, which can be specified in days only.
delay	This can be specified in minutes. After starting, Anacron waits for the 'n' number of minutes specified as the delay parameter, before actual job execution.
job-identifier	This is the name of the timestamps file and should be unique for each job. Before job execution, Anacron examines the timestamps file to decide whether job execution is needed or not.
command	This field, which is self-explanatory, can be a shell command or script.

Anacron provides a special string value that can be used in place of the period field. The currently supported value for this field is `@monthly`. This ensures that jobs will be executed once a month, regardless of the number of days in the current or previous month.

Anacron also allows the assignment of values to environment variables in the Anacron table. Please note that Anacron tables are parsed from top to bottom; hence, any environment settings are applicable only to those commands that are specified after setting environment variables. By default 'SHELL' is set to `/bin/`

`sh`. 'HOME' is the home directory of the user. 'LOGNAME' is the name of the user executing the Anacron job. The default value of 'PATH' is `/usr/bin:/bin`.

By default, after successful command execution, the output is mailed to the owner of the Anacron table (usually the root user). This default behavior can be overridden by setting up a 'MAILTO' environment variable. If 'MAILTO' is set up and is not empty, then the output of the command is mailed to the user so named. In 'MAILTO', multiple recipients can be specified by a comma separated list. If the empty value is assigned to 'MAILTO' (e.g., `MAILTO=""`), then no mail is sent.

In the Anacron table, we can assign values to environment variables in the same manner as shell assignment. The simple example given below will give you a clearer idea about the usage of environment variables.

```
[root]# cat /etc/anacrontab
# Set environment variables.
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
HOME=/root
LOGNAME=root
MAILTO="jerry@acme.com"
```

These replace Cron's entries

```
1 5 cron.daily run-parts --report /etc/cron.daily
7 10 cron.weeklyrun-parts --report /etc/cron.weekly
@monthly 15 cron.monthly run-parts --report /etc/cron.monthly
```

Additionally, Anacron provides two more variables (given below) that control the scheduled time of the configured jobs.

- 1) **RANDOM_DELAY**: This is the maximum number of minutes that will be added to the delay field of each job. The minimum possible value for this variable is 6 minutes. Here is an example. Let us suppose the value of **RANDOM_DELAY** is 20 minutes and the value in the delay field is 10 minutes. Then, before job execution, Anacron will wait for 10 minutes (from the delay field) + a random number of minutes between 6-20 (because of **RANDOM_DELAY**).
- 2) **START_HOURS_RANGE**: Anacron overcomes one of the major drawbacks of Cron. If an Anacron job is scheduled for a particular time interval and the system is not running at that time, Anacron guarantees that the job will be executed when the system comes up. But here is a catch. What if the system does not go offline? When should the job be executed? The solution is to specify this range by defining the **START_HOURS_RANGE** variable. For example, if the value of **START_HOURS_RANGE** is '12-18', then Anacron jobs can be run between 12 a.m. and 6 p.m. If **START_HOURS_RANGE** is defined and that time interval is missed, for example, because of a power outage,